How a question becomes an answer

The low-level architecture of FactoryPilot, the administration console, how a user is onboarded, and the three system diagrams — drawn from the code as deployed.

Part I — Diagrams

1 · A request, end to end

Eight gates in order. Two of them return an answer without ever reaching SAP or a model, and every path — including both of those — writes exactly one audit row.

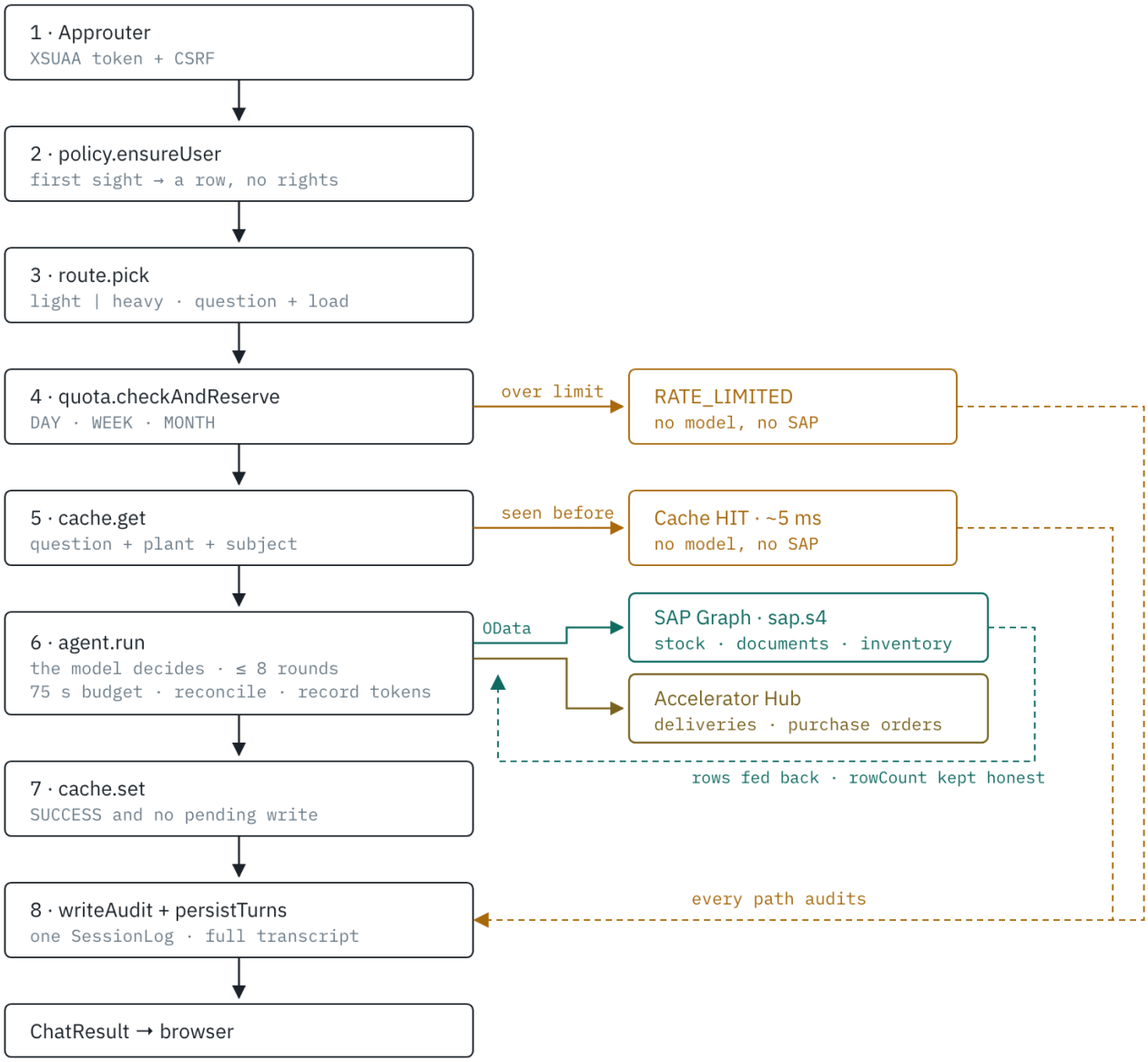

Figure 1 — a request, end to end.

2 · Who decides what

No code classifies the question. The model receives a tool catalogue built at request time from the active rows in `BusinessObjectConfig`, and chooses one of three things.

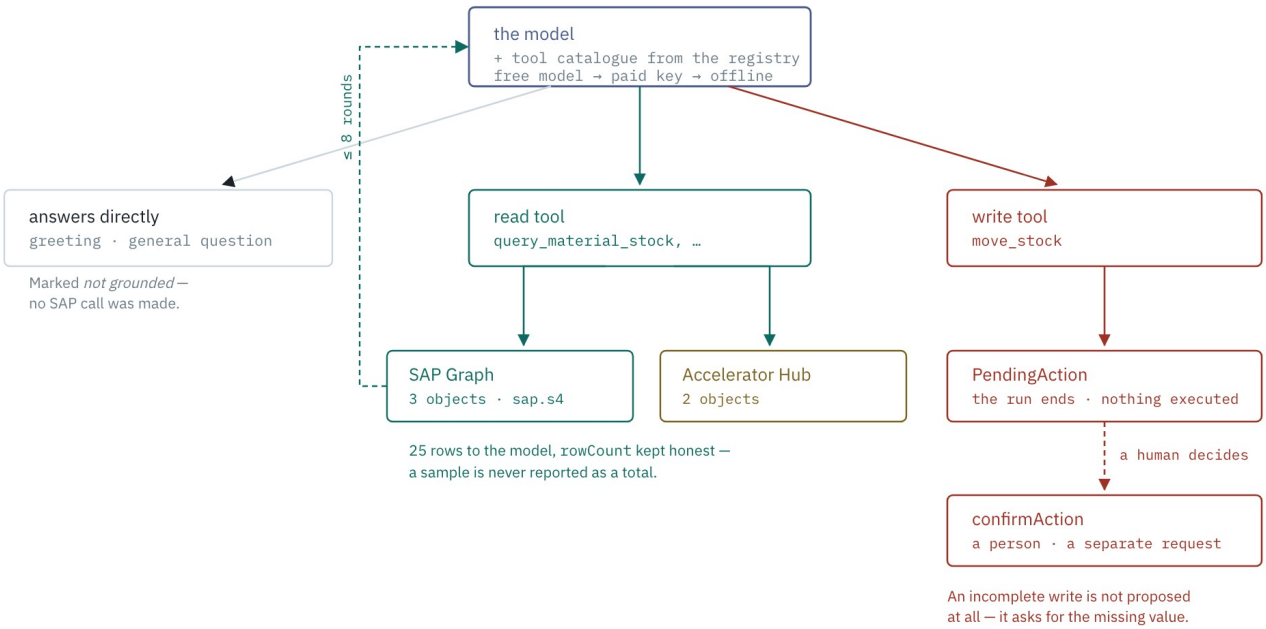

Figure 2 — who decides what inside the agent loop.

3 · The whole application

Every deployed part, what it binds to, and what leaves the tenant. One Node process runs eight CAP services behind one approuter; the agent loop is a library inside it, not a separate orchestrator.

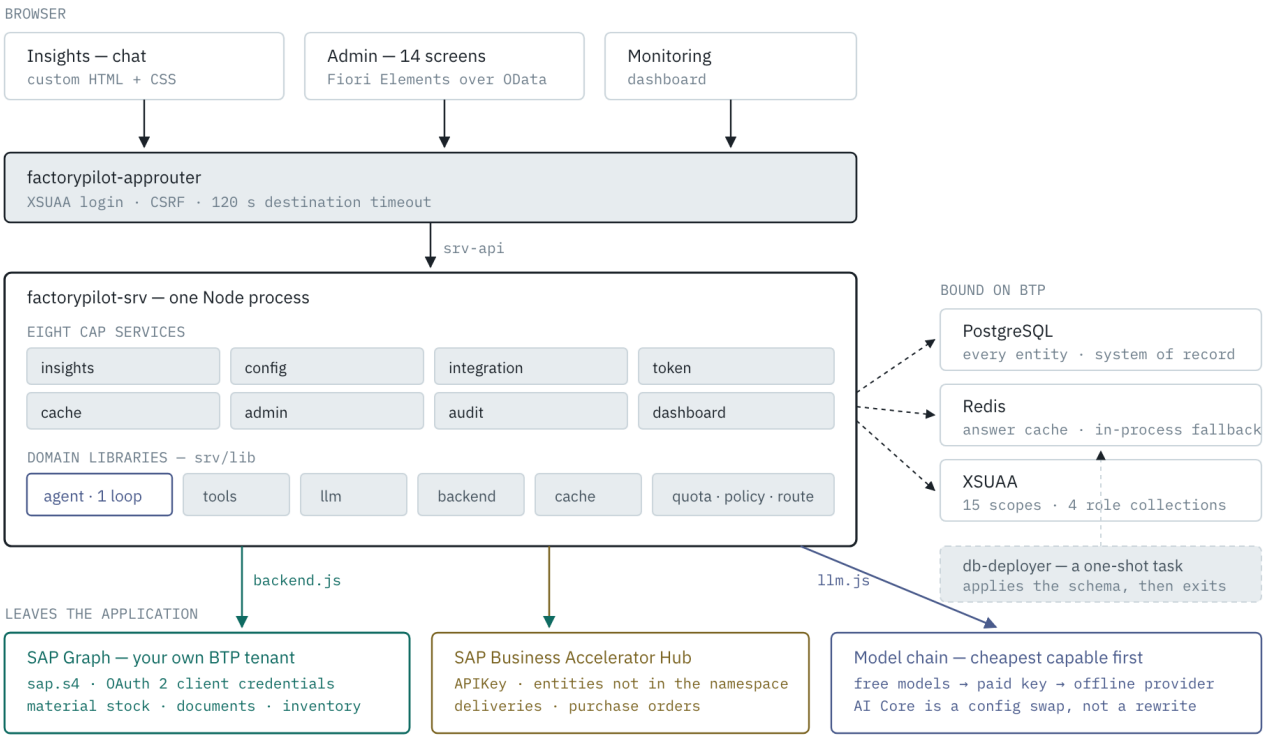

Figure 3 — the whole deployed application.

Drawn from branch version2 as deployed · 8 CAP services · 1 agent loop · 14 admin screens · 180 tests Colour is meaning, not decoration: teal is SAP Graph, ochre the Accelerator Hub, blue the model, amber an early exit, red a step only a person can take.

Part II — Architecture in detail

Every gate a request passes, every table it touches, and every way out — derived from the code as it stands, not from the design documents it was meant to follow.

1. The shape of it

One Node process runs eight CAP services behind one approuter. There is no separate orchestrator: the agent loop that decides which SAP object to query, and whether a write needs a human, runs inside the CAP service as a library (`srv/lib/agent.js`).

The thing that makes it a product rather than a script is that **what it can do is data, not code**. A business object is a row in `BusinessObjectConfig`; the endpoint it reads from is a row in `IntegrationEndpoint`; how long its answers live is a row in `CachePolicy`. Adding an SAP object is a row, not a release.

Layers

Layer	Lives in	Responsibility
Presentation	app/insights, app/admin, app/dashboard + 14 Fiori Elements apps	Chat, configuration, monitoring. Plain HTML for the three custom pages; Fiori Elements over OData for the CRUD screens.
Routing & auth	app/router (approuter)	XSUAA login, token exchange, forwards everything to srv-api. It does not enforce scopes — CAP does.
Services	srv/*-service.cds + .js	Eight OData v4 services. Scope enforcement via @restrict.
Domain logic	srv/lib/*.js	agent, tools, llm, backend, cache, quota, policy, oauth.
Persistence	db/*.cds	SQLite locally, PostgreSQL in production. 21 entities across 7 namespaces.

2. The request pipeline

This is the part that cannot be read off the file tree. `POST /insights/ask` passes six gates, and two of them can end the request early. The order matters: quota is reserved *before* the cache is consulted, so a cached answer still

costs a request against the limit — otherwise a user could pin one question and ask it forever for free.

The cache key

Composed from what changes the answer, and nothing else:

The warehouse is *in* the key rather than an afterthought: the same question against two plants is two answers, and merging them would show one site's figures to another. The key is computed **before** the run, from what is known up front — an earlier version keyed the write on the resolved `objectCode`, which no lookup could ever reproduce, so every request was a miss.

A question containing `today`, `now`, `current`, `this shift` or `so far` gets its TTL clamped to midnight, so "deliveries today" cannot survive into tomorrow regardless of the configured policy.

3. The agent loop

The model is given the tool catalogue and picks; the code does not classify intent. Tools are built at request time from active `BusinessObjectConfig` rows, so the catalogue is whatever an admin has registered.

settle() — why a run can end as FAILED

The model is handed `{"error": …}` as a tool result and will narrate around it. Before this gate existed, an unreachable warehouse endpoint produced *"No records matched that question"* — which tells a supervisor there is no stock, when nothing was ever checked.

A run that grounded nothing *and* had every tool call fail is `FAILED`, carries the reason, and is excluded from the cache by the `SUCCESS` guard — so a wrong answer cannot outlive the outage that produced it. A partial failure stays `SUCCESS`: real data was fetched, and the failed step is visible in `AgentStep`.

4. Reaching SAP

Every query leaves through one function, `backend.forEndpoint(endpoint)`, which picks an adapter from the endpoint's `kind`. The kind is data, so pointing a business object at a customer's own iFlow is a row change.

As deployed, three objects — material stock, material documents and physical inventory — read through **SAP Graph** on the customer's own SAP BTP tenant, authenticated by OAuth 2 client credentials. Deliveries and purchase orders read through the Accelerator Hub, because those entities are not in the Graph namespace. Material documents are the awkward one: Graph exposes headers, plant and material live on the items, so the query expands the association, filters inside the expansion and flattens the result to one row per item.

Credentials are never stored. `IntegrationEndpoint.credentialRef` holds the *name* of an environment variable. `CPI` expands to `CPI_CLIENT_ID` and `CPI_CLIENT_SECRET`, set with `cf set-env`. A seed validator rejects any `credentialRef` that looks like a secret rather than a name.

OAuth2 tokens are cached per endpoint until shortly before expiry. A `401` invalidates the cached token and retries exactly once — a token cached across a credential rotation is inside its stated expiry and still refused, and without

the retry every call fails until restart.

5. The eight services

One CDS service per capability, so a client can adopt or drop any of them independently. All are `@requires: 'authenticated-user'` at the service level, with per-entity `@restrict` splitting read from maintain.

Service	Path	Surface	Scopes
Insights	/insights	ask, confirmAction, health, Conversations, Messages	InsightsQuery
Config	/odata/config	BusinessObjects, ToolCatalog (view)	ConfigRead / ConfigMaintain
Integration	/odata/integration	Endpoints, EndpointTests, test()	IntegrationRead / IntegrationMaintain
Token	/odata/token	QuotaPolicies, Consumptions, TokenUsages, ModelRoutes, ApiKeyRefs, myUsage()	TokenRead / TokenMaintain
Cache	/odata/cache	CachePolicies, CacheStats, purge(), health()	CacheRead / CacheMaintain
Admin	/odata/admin	Users, UserScopes, ApprovalPolicies, OrgSettings, effectivePolicy(), canWrite()	AdminRead / AdminMaintain
Audit	/odata/audit	SessionLogs, AgentRuns, AgentSteps, PendingActions (read-only), Feedbacks	AuditRead
Dashboard	/odata/dashboard	overview(), quotaHeadroom()	DashboardRead / DashboardAdmin

Domain libraries

Module	Holds
agent.js	The tool loop, history sanitising, settle(). Max 8 rounds.
tools.js	Builds tool definitions from the registry; buildFilter templating; read/write execution.

llm.js	OpenRouterProvider, OpenAIProvider, AICoreProvider, FakeProvider behind one complete(), plus getProviderChain — free models first, the paid key next, offline last.
backend.js	Five SAP adapters — Graph, Hub, iFlow, CPI, mock — forEndpoint, OData v2/v4 parsing, expand flattening, fixture replay.
cache.js	Key composition, TTL resolution, midnight clamp, Redis with in-process fallback.
quota.js	Window arithmetic, conditional reservation, compensation.
policy.js	canWrite, ensureUser, policy merge, anomaly detection.
oauth.js	Client-credentials token cache, per-endpoint invalidation.

6. Data model

Twenty-one entities across seven namespaces, all prefixed `factorypilot`.

Namespace	Entities	Why it exists
config	BusinessObjectConfig	The registry. Service path, entity set, keywords, defaultFilters, selectFields, and the endpoint it reads from.
integration	IntegrationEndpoint, EndpointTest	Where data comes from: URL, kind, auth mode, credentialRef, timeout. Test results are kept so an admin can see when it last worked.
token	QuotaPolicy, Consumption, TokenUsage, ModelRoute, ApiKeyRef	Limits, rolling counters, per-call token records, and which model answers which class of request.
cache	CachePolicy, CacheStat	TTL and key scope per object; hit/miss/write counters for the dashboard.
admin	User, UserScope, ApprovalPolicy, OrgSettings	Who exists, which warehouses they may write to, and when a write needs a second person.
audit	SessionLog, AgentRun, AgentStep, PendingAction,	One SessionLog per request; the run and its individual

	Feedback	tool steps; proposed writes awaiting a human.
chat	Conversation, Message	Turn history, so a follow-up question has context.

Quota counters

Three rolling windows per subject — `DAY`, `WEEK` (Monday-start), `MONTH` — each one row in `Consumption`. The primary key is derived, not random:

With a random key, the first request of a period was a check-then-insert with a gap in the middle: eight concurrent callers each found no row and each created one, and a `SELECT.one` then saw one of eight. Deriving the key makes the database refuse the duplicate.

7. Identity and scopes

Identity comes from the XSUAA token; the approuter does not gate scopes, CAP does. Fifteen scopes are declared and bundled into four role collections assigned in the BTP cockpit.

Role collection	Intended for
FactoryPilot_BusinessUser	Asks questions, sees own usage.
FactoryPilot_ReadOnly	Read-only across configuration and logs.
FactoryPilot_ConfigAdmin	Registers business objects and endpoints.
FactoryPilot_Administrator	Everything, including users, quotas and approvals.

Two authorities, deliberately

OAuth scopes say what a caller may *call*. Warehouse write access is separate data — `UserScope` rows — because it is a business fact, not a platform one: a supervisor may write to plant 1000 and only read 1010.

Two rules keep that from deadlocking. A caller with no `User` row is provisioned on first question, **granted nothing** — no admin flag, no warehouse scope — so an administrator can see who is asking and decide. And a caller holding `AdminMaintain` is treated as an administrator from their token, because who administers the system is decided in BTP by whoever owns the subaccount.

Without the second rule, assigning yourself `FactoryPilot_Administrator` granted scopes but created no row, so `canWrite` refused every write — including for the person who administers the subaccount, whose only remedy was editing a table they could not reach through the product.

8. The administration console

Fourteen screens at `app/admin`, each a Fiori Elements application generated from the same CDS annotations that define the entity — so the list, its filters and its detail page all come from one description rather than three hand-written ones. The launchpad above them shows five live counts read straight from the services.

They divide cleanly in two, and the division is deliberate. **Configuration** decides what the system will do next and is editable. **Records** say what it already did and are not.

Configuration — an administrator changes these

Screen	Service	What it decides
Business Objects	/odata/config	The registry. Each active row becomes a tool the model may call — entity, filter template, fields to select, and a prompt hint telling the model how to talk about it. This is where a sixth SAP object is added, as a form.
Integration Endpoints	/odata/integration	Where data comes from: kind, URL, authentication mode, timeout. Holds the name of the environment variable carrying the secret, never the secret.
Cache Policies	/odata/cache	How long an answer may be reused before the question goes back to SAP. Per object, with a midnight clamp for anything time-bound.
Quota Policies	/odata/token	Day, week and month caps per user, per role, or DEFAULT. Also the per-request token ceiling and what happens on overage.
Model Routes	/odata/token	Which model serves light, heavy and intent traffic, and the fallbacks tried before the paid key is touched.
Users	/odata/admin	Identity and per-warehouse read/write scopes. Warehouse scopes are edited inside the user, as a composition.
Approval Policies	/odata/admin	When the agent may act unattended: per user, per warehouse, per organisation, with a write ceiling and a second-approver flag. Most restrictive layer wins.
Org Settings	/odata/admin	Organisation-wide defaults — default plant, the anomaly factor that decides when a quantity is unusual enough to

		need a human, and the digest hour.

Records — read-only on purpose

Screen	Service	What it shows
Session Log	/odata/audit	One row per request: who asked, what, which object answered, grounded or not, cache hit or miss, tokens spent, and the URL actually called.
Agent Runs	/odata/audit	A level below that — the rounds inside one turn, and every tool step underneath with its URL and duration.
Approvals	/odata/audit	The queue of proposed writes: pending, approved, rejected, expired. Nothing moves stock without passing through here.
Token Usage	/odata/token	One row per model call, flagging an estimated count as estimated rather than letting it read as measured.
Cache Effectiveness	/odata/cache	Hits, misses and tokens saved per object per day, so a TTL can be judged rather than guessed at.
Connection Tests	/odata/integration	Every connectivity check with status and duration — the screen for “it worked on Monday”.

Why the records are not editable. Making them so would be one annotation. It is left off deliberately: these tables are the evidence that a figure came from SAP, that a write was approved by a person, and that a quota was enforced. An audit trail someone can edit is not an audit trail, and the value of the whole design rests on being able to prove afterwards what happened.

How editing works

The configuration entities carry `@odata.draft.enabled`. Fiori Elements has no edit flow without it, which is why these screens were read-only until it was added — every list rendered and nothing could be changed.

It changes the OData contract, and that is worth knowing before writing a client: a direct `POST` creates a *draft*, not a row, and the row appears on activation. Every validation therefore guards `CREATE`, `UPDATE` *and* `SAVE` — without the last one a half-filled draft would activate into a record the rules would have refused, and the screen would cheerfully save a `GLOBAL` cache policy with no justification.

Scopes still decide. `AdminMaintain`, `ConfigMaintain`, `TokenMaintain`, `CacheMaintain` and `IntegrationMaintain` gate the writes exactly as before; a draft flow that quietly widened who may change a quota would be a worse bug than no edit flow at all.

9. Onboarding a user

Yes — a new person can be given access, a quota and warehouse rights, and be asking questions minutes later. The path crosses two systems, and the order matters for one specific reason explained below.

Where identity actually lives

FactoryPilot has no user store and no passwords. Identity is XSUAA: who someone is, and whether they may ask at all, is decided in the BTP cockpit. The `User` row in FactoryPilot is not an account — it is the record against which *warehouse* rights and a quota are attached, keyed by whatever `req.user.id` the token carries.

Step by step

- **BTP cockpit — add the person to the subaccount.** Security → Users → Create, with their email as the login. This is the only step that grants sign-in, and it is SAP's screen, not ours.
- **BTP cockpit — assign a role collection.** `FactoryPilot_BusinessUser` for someone who should ask questions and see their own usage. Without this they can sign in and will be refused at the first question, because `InsightsQuery` is what the service checks.
- **Have them sign in and ask one question.** Anything — "hello" is enough. On first sight `policy.ensureUser` creates their `User` row automatically. It grants nothing; it exists so the person becomes visible to whoever decides about them, rather than being discovered later in a log.
- **Admin → Users.** Their row is now there with the exact user id the token carries. Open it and add **warehouse scopes** — one row per plant, each `read` or `write`. Reading is already allowed by the role collection; a scope of `write` is what lets them approve a stock movement in that plant.
- **Admin → Quota Policies → Create.** Set `subject` to that same user id, then the daily, weekly and monthly caps. Save.
- **They ask again.** The new limits apply on the next question. Their own usage shows in the shellbar as `used / limit today`.

Step 3 before step 5, and here is why. A quota is matched on `subject` as an exact string against the user id in the token. Creating the `User` row by hand first means guessing that id — and BTP may give you the email, the login name or a GUID depending on how the identity provider is configured. Guess wrong and nothing errors: `ensureUser` quietly creates a *second* row on first sign-in, your warehouse scopes hang off the orphan, and the quota you typed never matches anyone. Letting the first question create the row means the id is right by construction, and steps 4 and 5 copy it rather than predict it.

Which quota applies

Resolution is deliberately dull, because the question that gets asked is "why was I blocked?" and it has to be answerable. The user's own row wins; failing that, a row whose subject matches one of their roles; failing that, `DEFAULT`. First match, no merging, no arithmetic across layers.

So a new person is covered by `DEFAULT` — 50 a day, 200 a week, 500 a month — from their very first question, whether or not anyone gets round to step 5. Giving them their own row is how you raise or lower that.

What each role collection gets them

Assign	They can	They cannot
FactoryPilot_BusinessUser	Ask questions; see their own token usage	See configuration, other people's history, or any admin screen
FactoryPilot_ReadOnly	Read every admin screen — config, tokens, users, audit, cache, integration	Change any of it, or ask a question
FactoryPilot_ConfigAdmin	Register business objects, manage endpoints and cache policy	Grant permissions or change quotas — wiring up a system is not the same job as deciding who may move stock
FactoryPilot_Administrator	Ask questions, configure, and administer	—

Writes need one more thing. The role collection decides whether someone may ask. Approving a goods movement additionally needs a `UserScope` row for that plant with `accessLevel = write`, or the `isAdmin` flag on the user. `policy.canWrite` checks exactly that, so a business user with no scopes can read every figure in a plant and still not move a pallet in it.

10. Deployment

One MTA, three modules, three managed services. Cloud Foundry on BTP; the Kyma target in `deploy/` is not the supported path.

Two environment switches change behaviour without a redeploy: `FACTORYPILOT_DEMO_MODE` replaces every backend with fixture replay, and `LLM_PROVIDER` overrides the configured `ModelRoute`.

11. Invariants

The properties the system is supposed to hold, and where each is enforced. Every one of these has been violated at some point during construction, which is why each has a test.

Invariant	Enforced by
Exactly one audit row per request, whatever the outcome	writeAudit on every exit path, including rate limits and cache hits
A write never executes inline	isWriteTool terminates the run with a PendingAction
An approved action runs exactly	Status transition guarded by WHERE status = 'PENDING'; a replay returns

once	ACTION_EXPIRED
A failed fetch is never an empty result	settle() marks the run FAILED, which also excludes it from the cache
A cap cannot be exceeded by concurrency	UPDATE … WHERE consumedCount <= limit − cost — the database decides, not a read-then-check
A cached answer never crosses users or plants improperly	Warehouse and the strategy's subject are in the key material
"Today" answers expire at midnight	TTL clamped for date-bound questions regardless of policy
Secrets never land in configuration	credentialRef holds a variable name; a seed validator rejects secret-shaped values
A new SAP object needs no code change	Tools are built at request time from active registry rows

Where it is verified

- **180 unit and service tests** — `cd apps/cap && npm test`
- **9 end-to-end scenarios** against a running instance — `node scripts/e2e.js`, also usable against a deployed URL
- **Pre-demo check** — `./scripts/demo-check.sh`: toolchain, seeds, fixtures, quota headroom, every question, every page
- **Hub probe** — `node scripts/hub-probe.js`: whether the field names the registry queries by exist upstream
- **Release gate** — `./scripts/release.sh`: refuses to tag unless the claim a tag makes is true

Verified since this was written. A Hub key and a Graph binding are both configured, and `node scripts/hub-probe.js` confirms every field named in `defaultFilters` and `selectFields` exists upstream. That check matters because OData does not fail loudly on a wrong filter column — it returns nothing, and the product then reports "no records matched" for data that is sitting there. Four of five objects had at least one wrong field name when this was first run.

Derived from the code on branch version2 · 8 CAP services · 21 entities · 15 scopes apps/cap/srv/lib is where the behaviour lives; db/*.cds is where the configuration surface is defined.